

Where the time goes at scale: a phase-resolved profile of FESOM2/Kokkos

Interim note, 6 August 2026 — Levante, 128-core CPU nodes. Per-rank per-phase wall clocks; wait is captured *inside* MPI by PMPI interposition, so busy = wall – wait is complete by construction. 115 timed steps.

The question. The communication-avoiding solvers cut the sea-surface-height (SSH) solve by 45–60%, yet the whole step improves by only 6–13%. If the SSH solve is the scaling bottleneck, why is the effect not larger, and where does the remaining time go?

1. The solve is the only part of the model that anti-scales. Splitting the CORE2 CPU ladder into the solve and everything else settles the premise:

ranks	step (ms)	SSH (ms)	rest (ms)	SSH speed-up	rest speed-up	ideal	ceiling	captured
128	195.50	4.89	190.61	1.00×	1.00×	1.0×	2.5%	29%
256	105.00	4.62	100.38	1.06×	1.90×	2.0×	4.4%	39%
432	66.20	6.22	59.98	0.79×	3.18×	3.4×	9.4%	61%
512	58.70	5.81	52.89	0.84×	3.60×	4.0×	9.9%	64%
864	43.50	8.22	35.28	0.59×	5.40×	6.8×	18.9%	69%

From 128 to 864 ranks everything except the solve speeds up 5.40× (79% parallel efficiency), while the solve becomes 41% *slower* in absolute terms. But it is still only 18.9% of the step at 864 ranks, which caps any whole-step gain from it: we measure –13.1%, or 69% of that ceiling, and the captured fraction rises monotonically with rank count. The lever works close to its limit; the limit is what is small.

2. Half the step is spent waiting inside MPI. Per-phase means, baseline solver (the phase total reconciles with the loop timer to 0.1 ms on both configurations):

phase	CORE2, 864 ranks		fArc, 2048 ranks		MPI calls
	busy (ms)	wait (ms)	busy (ms)	wait (ms)	per step
force	6.9	3.4	3.8	3.3	11
ice	0.2	0.9	0.6	0.8	9
icedyn	1.6	4.8	5.0	4.8	240
ocean	13.0	6.5	26.6	15.0	68
cg (SSH)	1.0	8.9	2.5	20.4	361 / 854
total	23.5	24.7	40.1	44.6	710 / 1203

(iceadv and other are omitted: 0.2–1.0 ms busy, ≤0.3 wait, and are in the totals.) The step is 48.2 ms on CORE2 and 84.7 on fArc, so wait is 51 and 53%: the model is synchronisation-bound. The solve is the largest single source of wait, yet roughly 90% of its cost *is* wait — its own arithmetic is 1.0 and 2.5 ms. The largest block of real computation is the ocean phase.

3. The saving is entirely wait, and two fifths of it appears in other phases. CORE2 at 864 ranks, baseline against pcsi in the same allocation:

phase	busy (ms)	wait (ms)	Δwait	MPI calls/step
force	6.9 → 6.5	3.4 → 1.5	–1.9	11 → 11
icedyn	1.6 → 1.6	4.8 → 3.6	–1.2	240 → 240
ocean	13.0 → 13.0	6.5 → 6.1	–0.4	68 → 68
cg	1.0 → 1.0	8.9 → 3.1	–5.8	361 → 170
total	23.5 → 23.1	24.7 → 15.2	–9.5	710 → 519

Compute time is unchanged, so the gain is entirely synchronisation. Only 5.8 of the 9.5 ms is inside the solver: 3.7 ms, or 39%, falls in phases whose MPI call counts and busy times are identical. Removing synchronisation points lets neighbouring phases stop absorbing skew, which is why the whole-step gain exceeds the solve’s own share of the step.

4. The larger target is load imbalance, not the solver. Compute time in the ocean phase spans 3.8/13.0/18.2 ms across CORE2 ranks and 8.8/26.6/42.5 across fArc ranks, a factor of 4.8 either way. A bulk-synchronous phase costs its maximum, not its mean, so this imbalance alone is 11% of the CORE2 step and **19% of the fArc step** — comparable to the entire solve and more than the solvers can win. It compounds too: the skew is absorbed at the next collective, the solve’s first reduction, so part of what the table attributes to solver wait is ocean imbalance arriving there.

5. Checking the tolerance less often is worth more than half the solver’s benefit. Chebyshev iteration needs no inner products, so P-CSI performs no reduction per iteration and its only global communication is the convergence test every K iterations. Sweeping K at CORE2 864 ranks (all legs **fallbacks=0**):

leg	s/step	d_{tot}	d_{SSH}	iters/solve
baseline cg	0.0432	—	—	105.90
pcsi, $K = 1$	0.0411	–4.86%	–20.12%	131.17
pcsi, $K = 5$ (default)	0.0384	–11.11%	–53.89%	133.35
pcsi, $K = 10$	0.0384	–11.11%	–57.22%	134.77
pcsi, $K = 20$	0.0379	–12.27%	–57.78%	142.27

Testing every fifth iteration rather than every one more than doubles the whole-step gain, –4.86 → –11.11%. Past five the return flattens and reappears as wasted work: $K = 20$ buys 1.2 pp more at 6.7% more iterations. Relatedly, that the solve is cheaper than the ice holds at the rank counts normally used but reverses here — the ice block costs 8.0 ms against the solve’s 9.9 on CORE2 and 11.8 against 22.9 on fArc, whereas at 128 ranks the solve is 2.5% of the step and the ice 5–13%.

Caveats. PMPI interposition adds overhead in proportion to call count, so these runs overstate the solver benefit (–20.5% here against the –13.1% of the timing protocol); the attribution, the busy figures and the ratios are what these numbers support, not the absolute deltas (§5 is free of this: it uses the timing protocol). Two configurations on one machine. METIS already weights by 2-D and 3-D node counts, so some ocean spread may be irreducible; that is untested, and the imbalance figure wants confirming at the 512-rank production point before it justifies re-partitioning.